Ballast for Source

Keep your tick afloat

Addon de stabilisation pour Garry's Mod — cadrage technique complet : limites du moteur, architecture, modules, multithreading, panel en jeu et exploitation d'un serveur de ~100 joueurs

Projet français créé par **Laki0s_** · version du document 0.1 · septembre 2026

Principe directeur : le thread principal est la ressource la plus précieuse du serveur. Chaque module de l'addon sert soit à mesurer ce qui le consomme, soit à le protéger, soit à en sortir du travail.

Identité du projet et conventions
Coding style Ballast (adapté d'Epitech)
0. Les limites qui cadrent tout
1. Architecture de l'addon
2. Module 1 : mesurer (profiler)
3. Module 2 : le scheduler
4. Module 3 : protection réseau
5. Module 4 : anti-lag physique
6. Module 5 : edicts et transmission
7. Module 6 : sortir du thread principal
8. Micro-optimisations Lua (LuaJIT)
9. Module 8 : côté client
10. Module 9 : monitoring et exploitation
11. Module 10 : panel en jeu
12. Ordre de développement et pièges
Code de conduite
Annexe : repères chiffrés

Identité du projet et conventions

Cette section fixe les noms à utiliser **partout** (code, Workshop, GitHub, documentation). Un nom unique et des préfixes cohérents évitent les conflits avec les autres addons et rendent le projet reconnaissable.

Identité

Élément	Valeur
Nom officiel	**Ballast for Source** (avec espaces, pour l'affichage : Workshop, GitHub, panel, docs)
Slogan	« Keep your tick afloat »
Description courte	Addon de stabilisation et d'optimisation serveur pour Garry's Mod
Créateur / mainteneur	**Laki0s_**
Origine	Projet français : documentation et communauté en français, code ouvert à l'international
Compatibilité	Garry's Mod uniquement (le nom évoque Source, mais l'addon repose sur le Lua et l'API de GMod)
Dépôt GitHub	`ballast-for-source`
Versionnage	SemVer : `MAJEUR.MINEUR.CORRECTIF`, départ en `0.1.0`

Noms techniques et préfixes

Élément	Convention	Exemple
Namespace Lua	Une seule table globale : `Ballast`	`Ballast.log(...)`
Module C++	Table globale séparée `BallastNative`	`BallastNative.Submit(...)`
Binaire C++	`gmsv_ballast_<os>.dll` dans `lua/bin`	`gmsv_ballast_linux64.dll`
Dossier Lua	`lua/ballast/` + préfixe de réalm	`lua/ballast/sv_net_guard.lua`
Autorun	Un seul point d'entrée	`lua/autorun/ballast_init.lua`
Identifiants de hooks	`Ballast_` + NomDuRôle	`Ballast_FrameTime`
Hooks personnalisés	`Ballast_` + événement	`Ballast_LagSpike`
Messages net	`Ballast_` + action	`Ballast_Sub`, `Ballast_Data`
Timers	`Ballast_` + rôle	`Ballast_PanelPush`
Convars	`ballast_` en minuscules	`ballast_phys_guard 1`, `ballast_debug 0`
Commandes	Console et chat	`ballast_panel`, `!ballast`
Fonctions, variables	snake_case anglais, verbe pour les fonctions	`Ballast.add_job`, `frame_time`
Constantes	UPPER_SNAKE_CASE	`LOG_MAX`, `DRAW_DIST`
Types C++	snake_case suffixé `_t`	`job_t`, `worker_state_t`
Clés de stats	snake_case en anglais	`props_frozen`, `net_dropped`
Fichiers de données	`data/ballast/`	`data/ballast/stats.json`

Initialisation de la table racine

```
Ballast = Ballast or {}
Ballast.name = "Ballast for Source"
Ballast.slogan = "Keep your tick afloat"
Ballast.version = "0.1.0"
Ballast.author = "Laki0s_"
Ballast.original = Ballast.original or {}
```

En-tête standard de chaque fichier

Format repris de l'en-tête Epitech (règle C-G1), adapté au projet. En Lua, on utilise `--` comme l'en-tête des Makefiles utilise `##` :

```
--
-- BALLAST FOR SOURCE PROJECT, 2026
-- ballast_for_source
-- File description:
-- Module 3 (Net Guard) : limite et valide les messages net entrants.
-- Auteur : Laki0s_ (contributeurs : voir CREDITS.md)
--
```

En C++, le même contenu dans un bloc `/* ** */`, comme dans les fichiers du module natif (section 7).

Conventions de code

- **Coding style** : tout le code C++ et Lua suit le **coding style Ballast**, adapté de l'Epitech C Coding Style (section suivante). C'est la référence en cas de doute.
- **Langue** : identifiants en anglais et en `snake_case` ; commentaires, documentation et messages de commit en français.
- **Textes affichés aux joueurs** : en français par défaut, centralisés dans `Ballast.lang` pour pouvoir ajouter l'anglais sans toucher au code.
- **Remplacement de fonctions de GMod** (`net.Incoming`, `hook.Add`) : toujours stocker l'original dans `Ballast.original` et prévoir un retour en arrière si le module est désactivé.
- **Chaque module** : une convar d'activation, des appels `Ballast.count` / `Ballast.log`, et une mesure avant/après documentée.
- **Hooks fréquents** : les règles de la section 8 (micro-optimisations) sont obligatoires, pas optionnelles.

Git et versions

- Branches : `main` (versions publiées, stables) et `dev` (développement) ; une branche par fonctionnalité (`feat/panel-modules`).
- Commits au format Conventional Commits, rédigés en français : `feat(netguard): limite par message`, `fix(panel): graphe vide au démarrage`, `perf(scheduler): budget adaptatif`.
- Chaque version publiée a son entrée dans `CHANGELOG.md`, avec les gains mesurés quand il y en a.

Coding style Ballast (adapté de l'Epitech C Coding Style)

Ballast adopte l'**Epitech C Coding Style (v8)** comme base, même si le projet est écrit en C++ et en Lua. Chaque règle est reprise avec son code d'origine (C-O1, C-F4…) pour qu'un étudiant ou un contributeur qui connaît le style Epitech s'y retrouve immédiatement, puis adaptée au langage

concerné. Quand une règle n'a pas de sens dans un langage, c'est indiqué « sans objet ».

Le principe reste celui d'Epitech : face à un choix, prendre l'option qui rend le code **plus clair, plus ergonomique et plus flexible**. Suivre le style dès le début coûte beaucoup moins cher que de corriger le code à la fin.

Niveaux de gravité appliqués à Ballast

Niveau	Signification dans Ballast
Fatale	Touche à la nature même du projet ou à la sécurité des serveurs. Une seule infraction : la contribution est refusée sans relecture.
Majeure	Touche à la structure du code. Correction obligatoire avant toute fusion.
Mineure	Touche à la présentation. À corriger ; des infractions répétées bloquent la fusion.
Info	Bonne pratique attendue. Les règles implicites (non écrites ici) sont de niveau info.

Comme chez Epitech, **toute tentative de contourner le coding style** (règle découpée artificiellement, désactivation des outils de vérification) entraîne le refus de la contribution. Et le style reste une convention de forme : il ne remplace jamais un code qui fonctionne et qui est mesuré.

C-O : organisation des fichiers

Règle	Gravité	C++ (module natif)	Lua (addon)
C-O1 Contenu du dépôt	**Majeure**	Aucun binaire (`.o`, `.obj`, `.dll`, `.so`) ni dossier `build/` : le binaire compilé est publié dans les Releases GitHub.	Aucun fichier temporaire (`*~`, `#*#`, `.bak`). `.gitignore` obligatoire.
C-O2 Extensions	**Majeure**	`.cpp` et `.hpp` uniquement.	`.lua` uniquement.
C-O3 Cohérence des fichiers	**Majeure**	Un fichier = une entité logique ; 10 fonctions maximum, dont 5 non-`static` maximum.	Un fichier = un module ; 10 fonctions maximum, dont 5 publiques (`Ballast.*`) maximum, les autres en `local function`.
C-O4 Nommage des fichiers	**Mineure**	snake_case anglais explicite : `worker.cpp`, pas `utils.cpp`.	snake_case anglais avec préfixe de réalm : `sv_net_guard.lua`. Exception : `gmsv_ballast_linux64.dll`, nom imposé par GMod.

C-G : portée globale

Règle	Gravité	C++ (module natif)	Lua (addon)
Extensions de langage	**Majeure**	Pas d'extension de compilateur : `[[noreturn]]` plutôt que `__attribute__`, include guards plutôt que `#pragma once`.	Pas de syntaxe propre à GMod : `!=`, `!`, `&&`, `||`, `//`, `/* */` et `continue` interdits. On écrit `~=`, `not`, `and`, `or`, `--`, `--[[ ]]`.
C-G1 En-tête de fichier	**Mineure**	Bloc `/* ** */` au format du projet, avec une vraie description.	Bloc `--` au même format (voir « En-tête standard »).
C-G2 Séparation des fonctions	**Mineure**	Une et une seule ligne vide entre deux fonctions.	Idem.
C-G3 Directives préprocesseur	**Mineure**	Indentées par 4 espaces selon l'imbrication, indépendamment du code.	Sans objet.
C-G4 Variables globales	**Majeure**	Uniquement des constantes (`const`, `constexpr`). Exception documentée : `native_state`, car l'API de GMod ne transmet aucun contexte aux fonctions natives.	Seules globales : `Ballast` et `BallastNative`. Le reste est `local`. Le remplacement de `net.Incoming` ou `hook.Add` est une exception documentée.
C-G5 include	**Majeure**	`#include` uniquement de fichiers d'en-tête.	`include()` et `AddCSLuaFile()` uniquement de fichiers `.lua` du projet.
C-G6 Fins de ligne	**Mineure**	LF uniquement, jamais de `\r`, jamais de `\` en fin de ligne.	Idem, imposé par `.gitattributes`.
C-G7 Espaces en fin de ligne	**Mineure**	Aucun.	Aucun.
C-G8 Lignes vides début/fin	**Mineure**	Aucune au début, une au maximum à la fin.	Idem.
C-G9 Valeurs constantes	**Mineure**	`constexpr` ou macro pour toute valeur non triviale.	Dans `Ballast.config` ou en constante `local` UPPER_SNAKE_CASE : aucun nombre magique.
C-G10 Assembleur inline	**Fatale**	Interdit.	Équivalent Lua, **fatal** aussi : aucune exécution de code dynamique (`RunString`, `CompileString`, contenu téléchargé).

C-F : fonctions

Règle	Gravité	C++ (module natif)	Lua (addon)
C-F1 Cohérence	**Mineure**	Une fonction fait une seule chose, à un seul niveau d'abstraction.	Idem : un hook appelle des fonctions nommées au lieu de tout contenir dans une fonction anonyme.
C-F2 Nommage	**Mineure**	snake_case anglais avec un verbe : `push_job`, `take_results`.	Idem : `Ballast.add_job`, `get_top_hooks`. Exception : noms imposés par GMod (`ENT:Draw`, `PANEL:Paint`, `net.Incoming`).
C-F3 80 colonnes	**Majeure**	80 colonnes maximum, fichiers de build compris.	80 colonnes maximum.
C-F4 20 lignes	**Majeure**	Corps de 20 lignes maximum.	Corps de 20 lignes maximum, fonctions anonymes des hooks comprises.
C-F5 4 paramètres	**Majeure**	4 maximum ; au-delà, regrouper dans une structure.	4 maximum ; au-delà, passer une table. `...` ne sert pas à contourner la limite.
C-F6 Fonctions sans paramètre	**Majeure**	En C++, `f()` et `f(void)` sont équivalents (la règle vise une ambiguïté propre au C) : on écrit `f()`.	Sans objet.
C-F7 Structures en paramètre	**Majeure**	Par pointeur ou référence `const`, jamais par copie.	Les tables sont déjà passées par référence : ne pas les copier pour les transmettre.
C-F8 Commentaires internes	**Mineure**	Aucun commentaire dans une fonction ; commentaire au-dessus du prototype.	Idem, avec un commentaire `---` au-dessus de la fonction.

C-L : mise en page dans une fonction

Règle	Gravité	C++ (module natif)	Lua (addon)
C-L1 Une instruction par ligne	**Majeure**	Pas d'affectations multiples, pas d'affectation dans une condition, pas de condition et d'instruction sur la même ligne.	Idem : pas de `a, b = 1, 2`, pas de `if x then return end` sur une ligne, pas de `net.Start() net.Send()` enchaînés. `local ok, err = pcall(f)` reste une seule instruction.
C-L2 Indentation	**Mineure**	4 espaces, aucune tabulation.	Idem.
C-L3 Espaces	**Mineure**	Un seul espace comme séparateur ; espace après virgule et mot-clé ; autour des opérateurs binaires ; aucun entre le nom d'une fonction et `(`.	Idem, y compris autour de `..` et `=`. Pas d'alignement avec plusieurs espaces.
C-L4 Blocs	**Mineure**	`{` en fin de ligne, sauf pour une définition de fonction où elle est seule ; `}` seule sauf `else`, `do while`, `struct`, `enum`.	`then` et `do` en fin de ligne ; `end` seul sur sa ligne ; `else` et `elseif` en début de ligne.
C-L5 Déclarations	**Majeure**	Au début de la fonction, une par instruction (sauf dans un `for`).	Tous les `local` au début de la fonction, un par ligne (sauf variables de `for`).
C-L6 Lignes vides	**Mineure**	Une ligne vide après les déclarations, aucune autre.	Idem.

C-V : variables et types

Règle	Gravité	C++ (module natif)	Lua (addon)
C-V1 Identifiants	**Mineure**	snake_case ; types suffixés `_t` ; macros, constantes et enums en UPPER_SNAKE_CASE.	snake_case ; constantes en UPPER_SNAKE_CASE. Exceptions : tables racines `Ballast` et `BallastNative` (nom du projet), API de GMod en PascalCase.
C-V2 Structures	**Majeure**	Petites et cohérentes.	Idem pour les tables : `Ballast.config`, `Ballast.stats`, `Ballast.logs` séparées plutôt qu'une table fourre-tout.
C-V3 Pointeurs	**Mineure**	`*` et `&` collés à l'identifiant de droite : `char *name`, `const job_t &job`, cast `(int *)ptr`.	Sans objet.

C-C : structures de contrôle

Règle	Gravité	C++ (module natif)	Lua (addon)
C-C1 Imbrication	**Majeure**	Profondeur de 3 ou plus interdite ; un `else if` compte pour 2.	Idem avec `elseif`. Solutions : retours anticipés, sous-fonctions, tables de dispatch.
C-C2 Ternaires	**Majeure**	Simples, valeur toujours utilisée, jamais imbriqués ni chaînés.	L'idiome `a and b or c` suit la même règle ; attention, il échoue si `b` vaut `false` ou `nil`.
C-C3 goto	**Majeure**	Interdit.	Interdit, ainsi que `continue` de GMod.

C-H : fichiers d'en-tête

Règle	Gravité	C++ (module natif)	Lua (addon)
C-H1 Contenu	**Majeure**	`.hpp` : prototypes, types, structures, enums, constantes, macros, fonctions `static inline`. Rien de tout cela dans les `.cpp`.	`sh_config.lua` joue ce rôle : constantes et configuration partagée uniquement, aucune logique.
C-H2 Include guard	**Majeure**	`#ifndef BALLAST_WORKER_HPP_` / `#define` / `#endif`, jamais `#pragma once`.	Chaque fichier n'est chargé qu'une fois, uniquement par `ballast_init.lua`.
C-H3 Macros	**Majeure**	Une instruction, une ligne ; préférer `constexpr` et les fonctions `inline`.	Sans objet.

C-A : règles avancées

Règle	Gravité	C++ (module natif)	Lua (addon)
C-A1 Constance	**Info**	`const` sur toute donnée non modifiée.	Pas de `const` en LuaJIT : une fonction ne modifie jamais une table reçue sans que son nom l'indique.
C-A2 Typage précis	**Info**	`size_t`, `uint32_t`, `int64_t`… selon l'usage.	Nombre de bits exact dans `net.WriteUInt` / `net.ReadUInt`.
C-A3 Saut de ligne final	**Info**	Obligatoire.	Obligatoire.
C-A4 static	**Info**	`static` pour tout ce qui ne sort pas du fichier.	`local` pour tout ce qui ne sort pas du fichier.

Exemple Lua conforme : le scheduler

Voici le module 2 réécrit selon le coding style : en-tête, fonctions courtes, aucune ligne compacte, déclarations en tête suivies d'une ligne vide, commentaires uniquement au-dessus des fonctions.

```
--
-- BALLAST FOR SOURCE PROJECT, 2026
-- ballast_for_source
-- File description:
-- Module 2 (Scheduler) : exécute les jobs avec un budget fixe par tick.
--

local SysTime = SysTime

--- Reprend un job et le retire de la file s'il est terminé.
local function resume_job(job)
    local ok, err = coroutine.resume(job)

    if not ok then
        ErrorNoHalt("[Ballast] " .. tostring(err) .. "\n")
    end
    if coroutine.status(job) == "dead" then
        table.remove(Ballast.jobs, 1)
    end
end

--- Exécute les jobs en attente sans dépasser le budget du tick.
local function run_jobs()
    local deadline = SysTime() + Ballast.config.job_budget

    while Ballast.jobs[1] and SysTime() < deadline do
        resume_job(Ballast.jobs[1])
    end
end

--- Ajoute une fonction à la file des jobs.
function Ballast.add_job(func)
    Ballast.jobs[#Ballast.jobs + 1] = coroutine.create(func)
end

hook.Add("Think", "Ballast_Scheduler", run_jobs)
```

L'exemple C++ de référence est le module natif de la section 7 (`worker.hpp`, `worker.cpp`, `module.cpp`).

Outils de vérification (l'équivalent de Banana)

Epitech vérifie son style avec l'outil Banana, réservé à ses étudiants. Ballast s'appuie sur des outils libres, lancés en local et dans la CI (GitHub Actions) à chaque pull request :

- **glualint** pour le Lua : indentation, espaces, syntaxe GMod non standard, profondeur d'imbrication.
- **clang-format** pour la mise en page C++ et **clang-tidy** pour les règles de structure.
- Une **relecture humaine** pour ce qu'aucun outil ne vérifie : cohérence des fonctions (C-F1), nommage explicite (C-F2), taille des structures (C-V2).
- Un petit **script maison** (optionnel) pour compter lignes par fonction, fonctions par fichier et paramètres, que les outils standards ne contrôlent pas tous.

native/.clang-format (base de configuration) :

```
BasedOnStyle: LLVM
IndentWidth: 4
UseTab: Never
ColumnLimit: 80
```

```
BreakBeforeBraces: Custom
BraceWrapping:
  AfterFunction: true
PointerAlignment: Right
SpaceBeforeParens: ControlStatements
AllowShortIfStatementsOnASingleLine: Never
AllowShortFunctionsOnASingleLine: None
AllowShortLoopsOnASingleLine: false
```

.gitattributes et **.gitignore** (règles C-G6 et C-O1) :

```
# .gitattributes
* text=auto eol=lf

# .gitignore
build/
*.o
*.obj
*.dll
*.so
*~
#*#
```

Les options exactes de `glualint.json` *(indentation de 4 espaces, syntaxe Lua standard, profondeur maximale) sont à reprendre depuis la documentation de glualint, car elles évoluent selon les versions.*

À propos des extraits de ce document

Les extraits des sections 2 à 11 illustrent des **principes d'optimisation** : certains gardent des commentaires internes ou des lignes compactes (`if x then return end`) pour rester lisibles dans un PDF. Le code du dépôt, lui, doit respecter strictement ce coding style. Les références à suivre sont le scheduler ci-dessus et le module natif de la section 7.

0. Les limites qui cadrent tout

Avant de coder, voici les contraintes que l'addon ne pourra jamais dépasser, seulement approcher.

- **Un seul thread principal** pour le Lua, la physique VPhysics, les entités et le réseau. L'état Lua n'est pas thread-safe.
- **Budget par tick** : `engine.TickInterval()` secondes, soit ~30 ms à 33 ticks et ~15 ms à 66. Tout doit tenir dedans.
- **8192 edicts maximum** : au-delà, crash « no free edicts ».
- **128 joueurs maximum**, avec une charge qui croît plus vite que le nombre de joueurs (chaque joueur reçoit l'état de toutes les entités qu'il voit et interagit avec les autres).
- **Client et serveur sont séparés** : optimiser les FPS client n'aide pas le serveur, et inversement.
- **Pas de changement de moteur possible** : le client attend un serveur Source 1 (protocole, maps, entités, physique). On ne peut qu'optimiser Source de l'intérieur (Lua, modules binaires, detours) et l'entourer de services externes. s&box (Source 2) est un autre jeu, les addons ne s'y transposent pas.

Ce que 100 joueurs impliquent concrètement

- **DarkRP / RP** : réaliste avec un vrai travail d'optimisation.

- **Sandbox libre** : très difficile à cause de la physique et des constructions (contraptions, Wiremod).
- **Modes à rounds** (TTT, Murder…) : possible techniquement, mais ces modes ne sont pas pensés pour autant de joueurs.

1. Architecture de l'addon

```
ballast/
├─ README.md / CHANGELOG.md / LICENSE
├─ CONTRIBUTING.md / CODE_OF_CONDUCT.md / CREDITS.md
├─ .gitignore / .gitattributes / glualint.json
├─ native/.clang-format                 -- style C++
├─ native/include/worker.hpp            -- types et prototypes
├─ native/src/worker.cpp                -- pool de threads
├─ native/src/module.cpp                -- liaison GMod
├─ lua/autorun/ballast_init.lua       -- chargement, AddCSLuaFile
├─ lua/ballast/sh_config.lua          -- seuils, activation par module
├─ lua/ballast/sv_profiler.lua        -- Module 1 : mesure
├─ lua/ballast/sv_scheduler.lua       -- Module 2 : répartition de charge
├─ lua/ballast/sv_net_guard.lua        -- Module 3 : protection réseau
├─ lua/ballast/sv_phys_guard.lua       -- Module 4 : anti-lag physique
├─ lua/ballast/sv_edicts.lua          -- Module 5 : budget d'entités et transmission
├─ lua/ballast/sv_async.lua           -- Module 6 : pont vers threads / services
├─ lua/ballast/cl_client.lua          -- Module 8 : optimisations client
├─ lua/ballast/sv_panel.lua           -- Module 10 : stats, journal, envoi au panel
├─ lua/ballast/cl_panel.lua           -- Module 10 : interface Derma en jeu
└─ (release) lua/bin/gmsv_ballast_linux64.dll   -- module C++ optionnel (niveau 3)
```

Chaque module doit pouvoir être **activé ou désactivé par une convar**, pour mesurer son effet isolément et le couper s'il entre en conflit avec un autre addon.

Configuration partagée (exemple)

```
Ballast = Ballast or {}
Ballast.config = {
    job_budget   = 0.002,          -- 2 ms de jobs max par tick
    edict_limit  = 7500,           -- refus des spawns au-delà
    net_max_bits  = 64 * 1024 * 8,  -- taille max d'un message entrant
    net_limits   = { ["mon_message"] = 10 }, -- messages / seconde / joueur
    net_default  = 30,
    spike_factor = 2,              -- pic = frame > 2 x TickInterval
    debug       = false,          -- active le profiler de hooks
}
```

2. Module 1 : mesurer (le profiler)

On n'optimise pas ce qu'on ne mesure pas. Deux mesures sont essentielles.

Temps de frame réel (détection des pics)

```
local last, budget = SysTime(), engine.TickInterval()
Ballast.frame_time = 0
hook.Add("Think", "Ballast_FrameTime", function()
    local now = SysTime()
    Ballast.frame_time = now - last
    last = now
    if Ballast.frame_time > budget * Ballast.config.spike_factor then
        hook.Run("Ballast_LagSpike", Ballast.frame_time)
    end
end)
```

Le hook `Ballast_LagSpike` sert de déclencheur aux autres modules (physique, scheduler).

Coût de chaque hook

On remplace `hook.Add` pour envelopper chaque fonction enregistrée et mesurer son temps avec `SysTime()`. On obtient un classement précis du type « le hook Think de tel addon coûte 4 ms ». Cette instrumentation a elle-même un coût : elle ne tourne qu'en **mode debug**. FProfiler reste la référence pour une analyse plus fine.

```
local raw_add = hook.Add
Ballast.hook_stats = {}
function hook.Add(event, id, fn)
    if not Ballast.config.debug or type(fn) ~= "function" then return raw_add(event, id, fn) end
    local key = event .. "/" .. tostring(id)
    local st = { total = 0, calls = 0, max = 0 }
    Ballast.hook_stats[key] = st
    return raw_add(event, id, function(...)
        local t = SysTime()
        local a, b, c, d, e, f = fn(...)
        local dt = SysTime() - t
        st.total, st.calls = st.total + dt, st.calls + 1
        if dt > st.max then st.max = dt end
        return a, b, c, d, e, f
    end)
end
```

*Attention : ce wrapper doit être chargé **avant** les autres addons (autorun au nom trié en premier) pour intercepter leurs hooks, et un `hook.Remove` reste compatible puisque l'identifiant ne change pas.*

À suivre également : `ents.GetEdictCount()`, `player.GetCount()` et la mémoire Lua via `collectgarbage("count")`.

3. Module 2 : le scheduler (étaler la charge)

C'est la pièce centrale. Au lieu qu'un calcul lourd prenne 20 ms d'un coup, on le découpe en coroutines exécutées avec un **budget fixe par tick**.

```
Ballast.jobs = {}
function Ballast.add_job(fn) Ballast.jobs[#Ballast.jobs + 1] = coroutine.create(fn) end

hook.Add("Think", "Ballast_Scheduler", function()
    local deadline = SysTime() + Ballast.config.job_budget
    while Ballast.jobs[1] and SysTime() < deadline do
        local co = Ballast.jobs[1]
        local ok, err = coroutine.resume(co)
        if not ok then ErrorNoHalt("[Ballast] " .. tostring(err) .. "\n") end
        if coroutine.status(co) == "dead" then table.remove(Ballast.jobs, 1) end
    end
end)
```

Les jobs appellent `coroutine.yield()` régulièrement, par exemple toutes les 50 entités traitées :

```
Ballast.add_job(function()
    local i = 0
    for _, ent in ents.Iterator() do
        -- traitement léger sur ent
        i = i + 1
        if i % 50 == 0 then coroutine.yield() end
    end
end)
```

- **Budget adaptatif** : si `Ballast.frame_time` est déjà élevé, on réduit le budget des jobs à ce tick.

- **Centraliser les timers** : remplacer des dizaines de `timer.Create` par un seul ticker qui dispatche.
- **Étaler plutôt que retarder** : un peu de travail à chaque frame vaut mieux qu'un gros bloc toutes les secondes.

4. Module 3 : protection réseau (net guard)

Beaucoup de crashs viennent de spam ou de messages malformés. Le point d'entrée du moteur est `net.Incoming` : on le réimplémente pour ajouter des limites **par joueur et par message**.

```
local counts = {}
function net.Incoming(len, ply)
    local name = util.NetworkIDToString(net.ReadHeader())
    if not name then return end
    name = name:lower()
    local func = net.Receivers[name]
    if not func then return end

    local c = counts[ply] or {}; counts[ply] = c
    c[name] = (c[name] or 0) + 1
    if c[name] > (Ballast.config.net_limits[name] or Ballast.config.net_default) then return end
    if len > Ballast.config.net_max_bits then return end

    func(len - 16, ply)
end
timer.Create("Ballast_NetReset", 1, 0, function() counts = {} end)
```

Règles côté envoi

- Bannir `net.WriteTable` (lent et lourd). Utiliser `net.WriteUInt` avec le nombre de bits minimal.
- Compresser les gros payloads avec `util.Compress`.
- N'envoyer qu'aux joueurs concernés : `net.Send(liste)` ou `net.SendPVS(pos)` plutôt que `net.Broadcast()`.
- Regrouper plusieurs petites mises à jour en un seul message par tick.
- Ne jamais appeler `SetNW*` en boucle : seulement quand la valeur change réellement.
- Valider chaque donnée lue dans un `net.Receive` (types, bornes, droits du joueur) : c'est la première source d'exploits.

5. Module 4 : anti-lag physique

La physique est le point le plus fragile à 100 joueurs. Trois niveaux de défense :

- **Réaction aux pics** : sur `Ballast_LagSpike`, geler tous les props non tenus par un joueur (`phys:EnableMotion(false)`).
- **Scan progressif** via le scheduler : parcourir les props par petits lots et geler ou passer en no-collide ceux qui s'interpénètrent (`phys:IsPenetrating()`) ou qui sont immobiles depuis longtemps.
- **Prévention** : gel automatique quelques secondes après le spawn, limites de props par joueur, refus des modèles trop gros (volume de l'OBB).

```
hook.Add("Ballast_LagSpike", "Ballast_PhysFreeze", function()
    for _, ent in ents.Iterator() do
        if ent:GetClass() == "prop_physics" and not ent:IsPlayerHolding() then
            local phys = ent:GetPhysicsObject()
            if IsValid(phys) and phys:IsMotionEnabled() then phys:EnableMotion(false) end
        end
```

```
    end
end)
```

En production, remplacer le parcours complet par le registre de props du module 5, beaucoup plus rapide.

6. Module 5 : budget d'entités et transmission

Edicts

Quand `ents.GetEdictCount()` dépasse le seuil configuré, on refuse les spawns via `PlayerSpawnProp`, `PlayerSpawnSENT`, `PlayerSpawnVehicle`, `PlayerSpawnNPC`, etc. (retour `false`). On nettoie aussi automatiquement les entités des joueurs déconnectés.

```
local function check_edict_budget(ply)
    if ents.GetEdictCount() > Ballast.config.edict_limit then
        ply:ChatPrint("Serveur plein en entités, spawn refusé.")
        return false
    end
end
for _, h in ipairs({ "PlayerSpawnProp", "PlayerSpawnSENT", "PlayerSpawnVehicle",
                     "PlayerSpawnNPC", "PlayerSpawnRagdoll", "PlayerSpawnEffect" }) do
    hook.Add(h, "Ballast_EdictGuard", check_edict_budget)
end
```

Transmission réseau

C'est un coût CPU souvent ignoré : pour chaque joueur, le moteur calcule quelles entités lui envoyer.

- Auditer les entités en `TRANSMIT_ALWAYS` et les repasser en `TRANSMIT_PVS` quand c'est possible.
- Faire du **culling par distance** avec `ent:SetPreventTransmit(ply, true)` pour les entités lointaines, recalculé périodiquement via le scheduler, jamais à chaque tick.
- Une map avec beaucoup d'occlusion (murs, bâtiments) réduit naturellement ce que chaque joueur voit.

Registres d'entités

Ne jamais appeler `ents.GetAll()` ou `ents.FindByClass()` dans un hook fréquent. On maintient ses propres tables :

```
Ballast.props = {}
hook.Add("OnEntityCreated", "Ballast_Registry", function(ent)
    timer.Simple(0, function()   -- la classe n'est pas toujours prête à la création
        if IsValid(ent) and ent:GetClass() == "prop_physics" then Ballast.props[ent] = true end
    end)
end)
hook.Add("EntityRemoved", "Ballast_Registry", function(ent) Ballast.props[ent] = nil end)
```

7. Module 6 : sortir du travail du thread principal

Règle absolue : ce qui lit ou modifie l'état du jeu (entités, physique, NW vars, hooks moteur) reste sur le thread principal. Tout le reste peut partir. Le hook lui-même ne peut pas être déplacé : c'est le moteur qui l'appelle sur son thread. Si on le faisait tourner ailleurs en attendant le résultat, aucun gain ; en vrai parallèle, crash assuré ; avec un mutex, tout redevient séquentiel. On **vide donc le hook** au lieu de le déplacer.

Classer les hooks

- **Réponse immédiate exigée** (`PlayerSay`, `CanTool`, `EntityTakeDamage`, `PlayerShouldTakeDamage`) : restent synchrones et ultra légers.
- **Sans réponse attendue** (`PlayerDeath`, `PlayerInitialSpawn`, `PlayerDisconnected`) : on copie les données et on déporte tout le reste.
- `Think` / `Tick` : impossibles à déplacer (c'est la boucle du jeu), mais un calcul lourd peut y être lancé en arrière-plan ou étalé via le scheduler.

Exemple : chat asynchrone

```
hook.Add("PlayerSay", "Ballast_AsyncChat", function(ply, text)
    Ballast.async.send("chat", { sid = ply:SteamID64(), text = text })
    return "" -- bloqué pour l'instant, rediffusé quand le résultat revient
end)

Ballast.async.on("chat", function(res)
    local ply = player.GetBySteamID64(res.sid)
    if IsValid(ply) and res.allowed then
        -- rediffuser res.text (filtré) ou exécuter la commande
    end
end)
```

Trois niveaux de déport

- **Niveau 1, sans module binaire** : `HTTP()` est déjà asynchrone, mysqloo aussi. Les écritures de fichiers et logs sont mises en tampon et vidées par lots via le scheduler.
- **Niveau 2, service externe** : un service en Node, Go ou Python relié par WebSocket (module gwsockets). Il gère logs, économie, filtrage du chat, stats. Un crash du service ne fait pas tomber le serveur.
- **Niveau 3, module C++ avec threads de travail** : le plus performant et le plus risqué.

Délais à prévoir

Source du délai	Ordre de grandeur
Attente du prochain tick	jusqu'à ~30 ms (33 ticks), ~15 ms (66 ticks)
Worker C++ dans le même processus	quasi nul + temps de calcul
Service local (WebSocket)	~1 ms + calcul
Service distant / API web	20 à 200 ms, parfois > 1 s
Total typique en local	15 à 50 ms
Total typique vers un service distant	50 à 300 ms

Invisible pour le chat, les commandes ou l'économie (< 100 ms). Inacceptable pour les dégâts, les tirs ou la physique : ces hooks restent synchrones. Ce délai s'ajoute au ping du joueur.

Module C++ (niveau 3), conforme au coding style

Le module est découpé en trois fichiers : un en-tête `.hpp` (types et prototypes), `worker.cpp` (le pool de threads, sans aucune dépendance à GMod) et `module.cpp` (la liaison avec le Lua). Chaque fonction fait moins de 20 lignes, sans commentaire interne, avec les déclarations en tête.

native/include/worker.hpp

```
/*
** BALLAST FOR SOURCE PROJECT, 2026
** ballast_for_source
** File description:
```

```
** Types et prototypes du pool de threads du module natif.
*/

#ifndef BALLAST_WORKER_HPP_
    #define BALLAST_WORKER_HPP_

    #include <atomic>
    #include <condition_variable>
    #include <mutex>
    #include <queue>
    #include <string>
    #include <thread>
    #include <vector>

typedef struct job_s {
    int id;
    std::string data;
} job_t;

typedef struct worker_state_s {
    std::queue<job_t> jobs;
    std::queue<job_t> results;
    std::mutex jobs_mutex;
    std::mutex results_mutex;
    std::condition_variable jobs_cv;
    std::atomic<bool> running{false};
    std::vector<std::thread> threads;
    int next_id = 0;
} worker_state_t;

std::string do_heavy_work(const std::string &input);
void start_workers(worker_state_t *state);
void stop_workers(worker_state_t *state);
int push_job(worker_state_t *state, const std::string &data);
std::queue<job_t> take_results(worker_state_t *state);

#endif /* BALLAST_WORKER_HPP_ */
```

native/src/worker.cpp (4 fonctions `static`, 4 publiques)

```
/*
** BALLAST FOR SOURCE PROJECT, 2026
** ballast_for_source
** File description:
** Pool de threads : file de jobs en entrée, file de résultats en sortie.
** Aucun appel à l'API Lua ici : tout est exécuté hors du thread principal.
*/

#include "worker.hpp"

static bool wait_for_job(worker_state_t *state, job_t *job)
{
    std::unique_lock<std::mutex> lock(state->jobs_mutex);

    state->jobs_cv.wait(lock, [state] {
        return !state->jobs.empty() || !state->running;
    });
    if (!state->running)
        return false;
    *job = std::move(state->jobs.front());
    state->jobs.pop();
    return true;
}

static void push_result(worker_state_t *state, job_t *job)
{
    std::lock_guard<std::mutex> lock(state->results_mutex);
```

```
    state->results.push(std::move(*job));
}

static void run_worker(worker_state_t *state)
{
    job_t job;

    while (wait_for_job(state, &job)) {
        job.data = do_heavy_work(job.data);
        push_result(state, &job);
    }
}

static void request_stop(worker_state_t *state)
{
    std::lock_guard<std::mutex> lock(state->jobs_mutex);

    state->running = false;
}

void start_workers(worker_state_t *state)
{
    unsigned int cores = std::thread::hardware_concurrency();
    unsigned int count = cores > 1 ? cores - 1 : 1;

    state->running = true;
    for (unsigned int i = 0; i < count; i++)
        state->threads.emplace_back(run_worker, state);
}

void stop_workers(worker_state_t *state)
{
    request_stop(state);
    state->jobs_cv.notify_all();
    for (std::thread &thread : state->threads)
        thread.join();
    state->threads.clear();
}

int push_job(worker_state_t *state, const std::string &data)
{
    std::lock_guard<std::mutex> lock(state->jobs_mutex);

    state->next_id++;
    state->jobs.push({state->next_id, data});
    state->jobs_cv.notify_one();
    return state->next_id;
}

std::queue<job_t> take_results(worker_state_t *state)
{
    std::lock_guard<std::mutex> lock(state->results_mutex);
    std::queue<job_t> taken;

    std::swap(taken, state->results);
    return taken;
}
```

native/src/module.cpp (liaison avec GMod)

```
/*
** BALLAST FOR SOURCE PROJECT, 2026
** ballast_for_source
** File description:
** Point d'entrée du module gmsv_ballast : expose BallastNative.submit
** et BallastNative.poll au Lua. Seule exception documentée à C-G4 :
** native_state, car l'API de GMod ne transmet aucun contexte.
*/
```

```
#include "GarrysMod/Lua/Interface.h"
#include "worker.hpp"

static worker_state_t native_state;

static void push_result_table(GarrysMod::Lua::ILuaBase *lua,
    std::queue<job_t> *results)
{
    lua->CreateTable();
    while (!results->empty()) {
        lua->PushNumber(results->front().id);
        lua->PushString(results->front().data.c_str(),
            static_cast<unsigned int>(results->front().data.size()));
        lua->SetTable(-3);
        results->pop();
    }
}

LUA_FUNCTION(submit_job)
{
    unsigned int length = 0;
    const char *input = nullptr;

    LUA->CheckType(1, GarrysMod::Lua::Type::String);
    input = LUA->GetString(1, &length);
    LUA->PushNumber(push_job(&native_state, std::string(input, length)));
    return 1;
}

LUA_FUNCTION(poll_results)
{
    std::queue<job_t> results = take_results(&native_state);

    push_result_table(LUA, &results);
    return 1;
}

GMOD_MODULE_OPEN()
{
    start_workers(&native_state);
    LUA->PushSpecial(GarrysMod::Lua::SPECIAL_GLOB);
    LUA->CreateTable();
    LUA->PushCFunction(submit_job);
    LUA->SetField(-2, "submit");
    LUA->PushCFunction(poll_results);
    LUA->SetField(-2, "poll");
    LUA->SetField(-2, "BallastNative");
    LUA->Pop();
    return 0;
}

GMOD_MODULE_CLOSE()
{
    stop_workers(&native_state);
    return 0;
}
```

`LUA` *est le nom imposé par les macros* `LUA_FUNCTION` *et* `GMOD_MODULE_OPEN` *de GMod : c'est une exception tolérée à C-V1. Dans nos propres fonctions, le paramètre s'appelle* `lua`*.*

Côté Lua, le pont (`sv_async.lua`) appelle `BallastNative.poll()` à chaque tick et déclenche les callbacks :

```
require("ballast")

local callbacks = {}
```

```
--- Envoie une chaîne au pool natif et mémorise le callback.
function Ballast.async.run(input, callback)
    local id = BallastNative.submit(input)

    callbacks[id] = callback
end

--- Déclenche le callback associé à un résultat.
local function dispatch_result(id, output)
    local callback = callbacks[id]

    callbacks[id] = nil
    if callback then
        callback(output)
    end
end

--- Récupère les résultats terminés depuis le dernier tick.
local function poll_native_results()
    for id, output in pairs(BallastNative.poll()) do
        dispatch_result(id, output)
    end
end

hook.Add("Think", "Ballast_Poll", poll_native_results)
```

Règles impératives du module C++

- Aucun appel à l'API Lua depuis un worker, jamais.
- Ne passer que des copies de données (strings, nombres, JSON), jamais de références à des objets du jeu.
- `join()` de tous les threads dans `GMOD_MODULE_CLOSE`, sinon crash au changement de map ou à l'arrêt.
- Laisser au moins un cœur libre pour le thread principal (voir `hardware_concurrency() - 1`), et ne pas lancer de workers sur le cœur où srcds est épinglé.
- Compiler en 64 bits pour la branche x86-64 (base : gmod-module-base), nommer le fichier `gmsv_ballast_linux64.dll` ou `gmsv_ballast_win64.dll`, le placer dans `garrysmod/lua/bin`, le charger avec `require("ballast")`.

Plus bas encore, des projets comme HolyLib modifient des fonctions internes du moteur (detours) pour en optimiser ou paralléliser certaines. C'est puissant mais fragile face aux mises à jour de GMod.

8. Micro-optimisations Lua (LuaJIT)

À appliquer dans tous les hooks fréquents, dans le moindre recoin.

- **Localiser les globales** en tête de fichier : `local IsValid, SysTime, CurTime = IsValid, SysTime, CurTime`.
- **Itérer sans allouer** : `player.Iterator()` et `ents.Iterator()` plutôt que `player.GetAll()`, qui crée une table à chaque appel.
- **Soulager le GC** : pas de création de tables, closures ou concaténations de strings dans `Think`. Réutiliser des tables vidées.
- **Distances au carré** : `a:DistToSqr(b) < r * r` plutôt que `Distance`.
- **Boucles numériques** (`for i = 1, #t`) plutôt que `pairs` dans les chemins critiques, mieux compilées par LuaJIT.

- **Throttler** : un hook qui n'a pas besoin de tourner à 33 Hz tourne à 5 Hz (test sur `CurTime()`) ou passe dans le scheduler.
- **Éviter les recherches spatiales répétées** (`ents.FindInSphere`, `ents.FindInBox`) à chaque tick : mettre en cache et rafraîchir périodiquement.
- **GC piloté** (avancé, à tester) : `collectgarbage("step", n)` pendant le temps libre du scheduler pour éviter les grosses pauses en pleine action.

```
local CurTime = CurTime
local next_run = 0
hook.Add("Think", "MonAddon_Throttle", function()
    local now = CurTime()
    if now < next_run then return end
    next_run = now + 0.2  -- 5 Hz au lieu de 33
    for _, ply in player.Iterator() do
        -- logique légère
    end
end)
```

9. Module 8 : côté client

Ça n'améliore pas le serveur, mais l'expérience des joueurs.

Proposer le rendu multicœur

Via un menu où le joueur accepte ou non, jamais de force : ces réglages font crasher certaines configurations.

```
gmod_mcore_test 1
mat_queue_mode 2
cl_threaded_bone_setup 1
cl_threaded_client_leaf_system 1
r_threaded_client_shadow_manager 1
r_threaded_particles 1
r_threaded_renderables 1
r_queued_ropes 1
studio_queue_mode 1
```

Aucun logiciel externe ne peut rendre multithread un programme qui ne l'est pas : Process Lasso et équivalents ne gèrent que la priorité et les cœurs attribués. Les outils qui s'injectent dans le jeu (DLL) exposent à un ban VAC.

Alléger les addons client

- Dans `HUDPaint`, aucun `surface.CreateFont` ni `Material()` : tout est créé une seule fois au chargement.
- `cam.Start3D2D` et les `ENT:Draw` coûteux sont ignorés au-delà d'une certaine distance.
- Limiter les `DynamicLight`, particules et effets.
- Réduire la taille du contenu à télécharger (collection Workshop, FastDL).

```
local DRAW_DIST = 800 * 800
function ENT:Draw()
    self:DrawModel()
    if LocalPlayer():GetPos():DistToSqr(self:GetPos()) > DRAW_DIST then return end
    cam.Start3D2D(self:GetPos() + self:GetUp() * 20, self:GetAngles(), 0.1)
        -- dessin coûteux uniquement de près
    cam.End3D2D()
end
```

10. Module 9 : monitoring et exploitation

L'addon envoie toutes les 10 secondes ses métriques via `HTTP()` vers Prometheus/Grafana (ou un simple endpoint) : temps de frame moyen et max, edicts, joueurs, mémoire Lua, messages net rejetés. On détecte ainsi une dérive avant le crash.

Configuration serveur (hors addon)

- Branche **x86-64** de srcds et `-tickrate 33` (ou 22 pour du RP calme).
- Serveur **dédié** avec CPU à forte performance mono-cœur, sous Linux (souvent en Docker). Pas de VPS partagé.
- srcds épinglé sur un cœur (`taskset`, voire `isolcpus`), gouverneur CPU en `performance`, priorité élevée.
- NVMe et RAM suffisante.
- Protection anti-DDoS, notamment contre le flood de requêtes A2S (iptables ou proxy).
- Watchdog de redémarrage automatique et restart quotidien à heure creuse.
- Hébergement proche des joueurs (datacenter en France) pour le ping.
- Architecture x86 uniquement : sur ARM, srcds nécessite Box64/FEX, plus lent et moins stable.

Au-delà de 128 joueurs

Le moteur ne suit plus : il faut plusieurs serveurs (instances) partageant une base de données commune, avec transfert des joueurs entre eux.

11. Module 10 : panel en jeu

Objectif : ouvrir en jeu une fenêtre qui résume l'état du serveur et **tout ce que l'addon a fait pour l'optimiser** (props gelés, messages réseau rejetés, spawns refusés, jobs exécutés, etc.). Le panel sert aussi de preuve : on voit en direct si un module est utile ou non.

Contenu du panel

Onglet	Contenu	Visible par
Vue d'ensemble	Temps de frame moyen/max, % du budget de tick, graphe des 60 dernières secondes, edicts, joueurs, mémoire Lua, jobs en attente	Admins
Actions	Journal horodaté des optimisations effectuées + compteurs cumulés depuis le démarrage	Admins
Hooks	Top 10 des hooks les plus coûteux (mode debug du profiler)	Admins
Modules	Activer / désactiver chaque module en direct	Superadmins
Mon client	FPS du joueur, état du rendu multicœur, bouton pour l'activer	Tous les joueurs

Étape 1 : chaque module enregistre ce qu'il fait

Une API commune (`sv_panel.lua`, chargée en premier) : des **compteurs cumulés** et un **journal circulaire** limité à 100 entrées pour ne jamais grossir en mémoire.

```
Ballast.stats = Ballast.stats or {}   -- ex. { props_frozen = 412, net_dropped = 37 }
Ballast.logs  = Ballast.logs  or {}   -- les plus récents en premier
local LOG_MAX = 100

function Ballast.count(key, n)
    Ballast.stats[key] = (Ballast.stats[key] or 0) + (n or 1)
end

function Ballast.log(cat, msg)
    table.insert(Ballast.logs, 1, { t = os.time(), cat = cat, msg = msg })
```

```
    Ballast.logs[LOG_MAX + 1] = nil
end
```

Exemples d'appels à ajouter dans les modules existants :

```
-- Module 4 (physique), après un gel sur pic de lag
Ballast.count("props_frozen", n)
Ballast.log("Physique", n .. " props gelés (pic de " .. math.Round(ft * 1000) .. " ms)")

-- Module 3 (net guard), quand un message est rejeté
Ballast.count("net_dropped")
Ballast.log("Réseau", ply:Nick() .. " : " .. name .. " rejeté (spam)")

-- Module 5 (edicts), quand un spawn est refusé
Ballast.count("spawns_refused")

-- Module 2 (scheduler), à chaque job terminé
Ballast.count("jobs_done")
```

Pour les actions très fréquentes (ex. un message rejeté 500 fois par seconde), on ne journalise qu'une ligne agrégée par seconde, sinon le journal perd son sens et consomme du CPU.

Étape 2 : mesurer moyenne et maximum sur la seconde

Le panel affiche une moyenne et un pic par seconde, plus lisibles qu'une valeur instantanée. On complète le hook du module 1 :

```
local acc, count, peak = 0, 0, 0
Ballast.history = {}          -- 60 dernières secondes, pour le graphe
-- dans le hook Think "Ballast_FrameTime", après le calcul de Ballast.frame_time :
acc, count = acc + Ballast.frame_time, count + 1
if Ballast.frame_time > peak then peak = Ballast.frame_time end

timer.Create("Ballast_FrameStats", 1, 0, function()
    Ballast.frame_avg = count > 0 and acc / count or 0
    Ballast.frame_max = peak
    table.insert(Ballast.history, Ballast.frame_avg)
    if #Ballast.history > 60 then table.remove(Ballast.history, 1) end
    acc, count, peak = 0, 0, 0
end)
```

Étape 3 : envoyer les données seulement aux panels ouverts

Le panel ne doit rien coûter quand personne ne le regarde. Le client **s'abonne** à l'ouverture et se désabonne à la fermeture ; le serveur n'envoie qu'une fois par seconde, qu'aux abonnés, et vérifie les droits.

```
util.AddNetworkString("Ballast_Sub")
util.AddNetworkString("Ballast_Data")
local subs = {}

net.Receive("Ballast_Sub", function(_, ply)
    if not ply:IsAdmin() then return end          -- vérification côté serveur, toujours
    subs[ply] = net.ReadBool() or nil
end)
hook.Add("PlayerDisconnected", "Ballast_Sub", function(ply) subs[ply] = nil end)

timer.Create("Ballast_PanelPush", 1, 0, function()
    if not next(subs) then return end              -- personne ne regarde : coût nul
    local to = {}
    for ply in pairs(subs) do if IsValid(ply) then to[#to + 1] = ply end end

    local raw = util.TableToJSON({
        avg = Ballast.frame_avg, max = Ballast.frame_max, budget = engine.TickInterval(),
        hist = Ballast.history, edicts = ents.GetEdictCount(), players = player.GetCount(),
        mem = collectgarbage("count"), jobs = #Ballast.jobs,
        stats = Ballast.stats, logs = Ballast.logs, hooks = Ballast.get_top_hooks(10),
```

```
    })
    local data = util.Compress(raw)
    net.Start("Ballast_Data")
        net.WriteUInt(#data, 16)
        net.WriteData(data, #data)
    net.Send(to)
end)
```

`Ballast.get_top_hooks(n)` trie `Ballast.hook_stats` (module 1) par temps total et renvoie les n premiers. Ici le JSON compressé est acceptable : un seul envoi par seconde, à quelques admins. Amélioration possible : n'envoyer que les nouvelles lignes du journal depuis le dernier envoi.

Étape 4 : l'interface Derma (client)

```
concommand.Add("ballast_panel", function() Ballast.open_panel() end)
hook.Add("OnPlayerChat", "Ballast_Cmd", function(ply, txt)
    if ply == LocalPlayer() and txt:lower() == "!ballast" then Ballast.open_panel() return true end
end)

net.Receive("Ballast_Data", function()
    local len = net.ReadUInt(16)
    local json = util.Decompress(net.ReadData(len)) or ""
    Ballast.data = util.JSONToTable(json)
    if IsValid(Ballast.panel) and Ballast.data then Ballast.panel:refresh_data(Ballast.data) end
end)

local function set_subscription(state)
    net.Start("Ballast_Sub") net.WriteBool(state) net.SendToServer()
end

function Ballast.open_panel()
    if IsValid(Ballast.panel) then Ballast.panel:Remove() end
    local f = vgui.Create("DFrame")
    f:SetSize(ScrW() * 0.5, ScrH() * 0.6) f:Center() f:MakePopup()
    f:SetTitle("Ballast : état du serveur et optimisations")
    Ballast.panel = f

    local sheet = vgui.Create("DPropertySheet", f) sheet:Dock(FILL)
    if LocalPlayer():IsAdmin() then
        set_subscription(true)
        f.OnRemove = function() set_subscription(false) end
        sheet:AddSheet("Vue d'ensemble", Ballast.build_overview(sheet), "icon16/chart_line.png")
        sheet:AddSheet("Actions", Ballast.build_actions(sheet), "icon16/lightning.png")
        sheet:AddSheet("Hooks", Ballast.build_hooks(sheet), "icon16/time.png")
    end
    if LocalPlayer():IsSuperAdmin() then
        sheet:AddSheet("Modules", Ballast.build_modules(sheet), "icon16/cog.png")
    end
    sheet:AddSheet("Mon client", Ballast.build_client(sheet), "icon16/monitor.png")

    function f:refresh_data(d)
        for _, tab in ipairs(sheet:GetItems()) do
            if tab.Panel.refresh_data then tab.Panel:refresh_data(d) end
        end
    end
end
```

Le graphe du temps de frame

Un panel personnalisé qui dessine l'historique, avec une ligne rouge pour le budget du tick. Le dessin est très léger (60 segments) et ne tourne que panel ouvert.

```
function Ballast.build_graph(parent)
    local g = vgui.Create("DPanel", parent)
    g:SetTall(160)
    function g:Paint(w, h)
        draw.RoundedBox(4, 0, 0, w, h, Color(25, 28, 34))
```

```
        local d = Ballast.data
        if not d or not d.hist or #d.hist < 2 then return end
        local scale = h / (d.budget * 2)           -- l'échelle monte à 2 x le budget
        local by = h - d.budget * scale
        surface.SetDrawColor(220, 60, 60)          -- ligne du budget
        surface.DrawLine(0, by, w, by)
        surface.SetDrawColor(90, 200, 120)
        local step = w / (#d.hist - 1)
        for i = 2, #d.hist do
            surface.DrawLine((i - 2) * step, h - math.min(d.hist[i - 1] * scale, h),
                             (i - 1) * step, h - math.min(d.hist[i] * scale, h))
        end
    end
    return g
end
```

L'onglet Actions

```
local LABELS = {
    props_frozen = "Props gelés", net_dropped = "Messages réseau rejetés",
    spawns_refused = "Spawns refusés (edicts)", jobs_done = "Jobs exécutés",
}
function Ballast.build_actions(parent)
    local p = vgui.Create("DPanel", parent) p:SetPaintBackground(false)
    local totals = vgui.Create("DLabel", p) totals:Dock(TOP) totals:SetTall(60)
    totals:SetWrap(true)
    local list = vgui.Create("DListView", p) list:Dock(FILL)
    list:AddColumn("Heure"):SetFixedWidth(70)
    list:AddColumn("Catégorie"):SetFixedWidth(90)
    list:AddColumn("Action effectuée")

    function p:refresh_data(d)
        local parts = {}
        for k, v in pairs(d.stats or {}) do
            parts[#parts + 1] = (LABELS[k] or k) .. " : " .. v
        end
        totals:SetText(table.concat(parts, "   |   "))
        list:Clear()
        for _, e in ipairs(d.logs or {}) do
            list:AddLine(os.date("%H:%M:%S", e.t), e.cat, e.msg)
        end
    end
    return p
end
```

L'onglet Modules (superadmins)

Chaque module est piloté par une convar serveur (`ballast_phys_guard 1`, etc.). Le panel affiche une case à cocher par module et envoie le changement via un message `Ballast_Toggle`. Le serveur **revérifie** `ply:IsSuperAdmin()` et que le nom fait partie d'une liste blanche avant d'appliquer, puis journalise le changement (« Admin X a désactivé le module Physique »).

L'onglet Mon client (tous les joueurs)

Entièrement local, sans réseau : FPS du joueur (moyenne de `1 / RealFrameTime()`), état de `gmod_mcore_test` et des autres réglages multicœur, et un bouton « Activer le rendu multicœur » qui applique les convars via `RunConsoleCommand` après confirmation, avec un avertissement sur les crashs possibles.

Règles pour que le panel n'annule pas les gains

- Aucun envoi si aucun panel n'est ouvert, et 1 Hz maximum.
- Données réservées aux admins, avec vérification **côté serveur** (jamais se fier au client).
- Ajouter `Ballast_Sub` et `Ballast_Toggle` aux limites du net guard (ex. 5 par seconde).

- Journal borné (100 lignes) et agrégé pour les événements répétitifs.
- Rafraîchir les listes seulement à la réception des données, jamais dans `Paint` ou `Think`.
- Garder le payload sous 64 Ko (limite d'un message net) : si le journal grossit, envoyer seulement le delta.

12. Ordre de développement et pièges

Ordre conseillé (meilleur rapport gain/effort d'abord)

1. Profiler : sans lui, tout le reste est à l'aveugle.
2. Net guard : stabilité et sécurité immédiates.
3. Panel en jeu (au moins l'onglet Vue d'ensemble et le journal) : il permet de voir l'effet de chaque module dès son ajout.
4. Physique et edicts : les crashs les plus fréquents.
5. Scheduler, puis migration des addons existants dessus.
6. Micro-optimisations des hooks identifiés par le profiler.
7. Déport asynchrone (niveaux 1 et 2).
8. Module C++, seulement si le profiler montre un calcul lourd qui le justifie.

Pièges à anticiper

- Remplacer `hook.Add` ou `net.Incoming` peut entrer en conflit avec d'autres addons (ULX, anti-cheats, gamemode) qui font de même. Gérer l'ordre de chargement et garder un fallback vers la fonction d'origine.
- Chaque module doit être désactivable, et chaque changement mesuré avant/après.
- Le profiler en continu coûte lui-même du temps de frame.
- Des limites réseau trop strictes cassent des addons légitimes : commencer en mode « log seulement », puis activer le blocage.
- Un module C++ mal fermé ou qui touche au Lua depuis un worker fait crasher le serveur de façon aléatoire : le pire bug possible à diagnostiquer.

Code de conduite

Ballast for Source est un projet communautaire français créé par **Laki0s_**. Ce code de conduite s'applique à tous les espaces du projet : dépôt GitHub, Discord, commentaires Workshop, issues et pull requests. Il est à copier dans `CODE_OF_CONDUCT.md`.

1. Respect et bienveillance

- On critique le code, jamais les personnes. Les retours sont précis, argumentés et constructifs.
- Aucune insulte, moquerie, discrimination ni harcèlement, quel qu'en soit le motif.
- Les débutants sont les bienvenus : on explique plutôt que de juger.
- Le français est la langue principale du projet ; l'anglais est accepté pour que chacun puisse participer.

2. Engagements de confiance envers les serveurs

Un addon serveur a un accès total au serveur et aux joueurs. Ballast s'engage donc à :

- **Aucune backdoor** : aucun accès caché, aucun privilège réservé au créateur ou aux contributeurs.
- **Aucun code obfusqué** : tout le code est lisible et vérifiable.
- **Aucune exécution de code distant** : jamais de `RunString` ou `CompileString` sur du contenu téléchargé.
- **Aucune collecte cachée** : les métriques restent sur le serveur ou vont uniquement vers l'endpoint configuré par l'administrateur. Aucune donnée de joueur (SteamID, IP, chat) n'est envoyée à des tiers.
- **Aucune fonctionnalité cachée** : tout ce que fait l'addon est visible dans le panel et le journal.
- **Respect des joueurs** : aucun réglage n'est forcé côté client ; le rendu multicœur est proposé, jamais imposé.

3. Sécurité et divulgation responsable

- Une faille (exploit net, crash, contournement de droits) se signale **en privé** à Laki0s_, jamais publiquement en premier.
- Le mainteneur accuse réception, corrige, puis publie un correctif avec crédit au découvreur (s'il le souhaite).
- Il est interdit d'exploiter une faille de Ballast sur un serveur tiers.

4. Règles de contribution

- Toute contribution passe par une pull request relue avant fusion.
- Une optimisation doit être **mesurée** : chiffres avant/après (temps de frame, mémoire, edicts) dans la description.
- Les conventions de nommage et de code de ce document s'appliquent à tous.
- Chaque contributeur est crédité dans `CREDITS.md`.
- Le créateur **Laki0s_** reste crédité comme auteur original dans tout fork ou redistribution, selon les termes de la licence.

5. Modération

Laki0s_ (et les modérateurs qu'il désigne) fait respecter ce code. Les sanctions sont graduées : rappel à l'ordre, avertissement, exclusion temporaire, puis exclusion définitive. Un comportement grave (harcèlement, menaces, diffusion de code malveillant) peut mener directement à l'exclusion.

6. Licence (à choisir)

La licence définit ce que les autres ont le droit de faire avec le code. Deux options courantes pour un addon open source :

- **MIT** : très permissive. Tout le monde peut réutiliser, modifier et redistribuer, y compris dans des projets fermés, en gardant la mention de l'auteur.
- **GPL-3.0** : les versions modifiées et redistribuées doivent rester open source sous la même licence, ce qui protège contre les reprises fermées ou revendues.

Quelle que soit la licence choisie, elle se place dans `LICENSE` et se rappelle dans l'en-tête de chaque fichier. En cas de doute sur un point juridique précis, mieux vaut consulter un juriste.

Annexe : repères chiffrés

Élément	Valeur / repère
Budget par tick à 66 / 33 / 22 ticks	~15 ms / ~30 ms / ~45 ms
Limite d'edicts GMod	8192 (seuil de sécurité conseillé : ~7500)
Joueurs max par serveur	128
Budget conseillé du scheduler	1 à 3 ms par tick
Seuil de pic de lag	frame > 2 × TickInterval
Délai asynchrone local / distant	15-50 ms / 50-300 ms
Délai imperceptible (chat, économie)	< 100 ms
Fréquence d'envoi des métriques	toutes les 10 s